A Project Report On

# MediRemind

**AI Based Medicine Reminder System**

Submitted in Partial Fulfillment of

**Bachelor of Science in Computer Science (2023–2027)**

Course: **Artificial Intelligence**

Instructor: **Sir Faisal Hafeez**

Course: **Mobile Application Development**

Instructor: **Ma'am Nabiha Komal**


Submitted by

**Kaneez Fatima**

BSCS-M-A-23-09

UNIVERSITY OF LAYYAH 2022

**Department of Computer Science**

University of Layyah, Main Campus Hafizabad

# Acknowledgement

I would like to express my sincere gratitude to my respected instructors, **Sir Faisal Hafeez** and **Ma'am Nabiha Komal**, for their continuous guidance, support, and encouragement throughout the development of this project. Their valuable feedback and teaching helped me understand both the theoretical and practical aspects required to build this application successfully.

I am also thankful to the **Department of Computer Science** for providing a productive learning environment and the necessary resources for completing this work.

*Regards,*

**Kaneez Fatima**

BSCS-M-A-23-09

# Abstract

**MediRemind** is an Android-based medicine reminder application designed to help users manage their medication schedules efficiently and accurately. The application allows users to scan handwritten or printed prescriptions, extract medicine details through OCR and AI-based processing, and automatically generate reminder alarms based on predefined scheduling rules.

The system supports manual medicine entry for cases where prescription scanning is not required. All extracted and manually entered data is persisted in a local **SQLite** database. Alarms are scheduled using Android's **AlarmManager** API to ensure reliable, on-time reminders that function even when the application is closed.

The project is developed using **Java** and **XML** in Android Studio. It integrates core concepts from both *Mobile Application Development* and *Artificial Intelligence*, demonstrating a practical, real-world implementation of AI-assisted prescription parsing and intelligent alarm generation on a mobile platform.

## Contents

# 1. Introduction

## 1.1 Overview

In today's fast-paced lifestyle, people often forget to take their medicines on time, leading to missed doses, incorrect schedules, and serious health consequences. To address this real-world problem, **MediRemind** was developed, an Android-based intelligent medicine reminder system that combines OCR technology and AI-driven prescription parsing with a robust alarm scheduling engine.

## 1.2 Problem Statement

Managing multiple medicines manually is difficult and error-prone. Users may forget timings, misread handwritten prescriptions, or lose track of dose frequencies. Existing generic reminder apps do not understand medical prescriptions or generate schedule-aware alarms. There is a clear need for a mobile solution that can intelligently parse prescriptions and generate accurate, medication-specific reminders.

## 1.3 Objectives

- To allow users to scan prescriptions and extract medicine details using OCR and AI techniques.
- To support manual medicine entry as an alternative input method.
- To generate accurate, rule-based alarms from medicine frequency and timing information.
- To provide a clean, user-friendly interface for managing medicine schedules and viewing alarm history.
- To ensure alarms work reliably even after device reboot.

## 1.4 Scope of the Project

MediRemind targets individual Android users who need a personal medicine management tool. The application focuses on prescription scanning, alarm generation, and schedule tracking. It is designed for use in Urdu-English bilingual prescription contexts, with AI

logic capable of interpreting both scripts.

## 2. System Overview

### 2.1 System Description

MediRemind is a native Android application built in Java. It integrates a cloud-based OCR engine (PaddleOCR) and a large language model (Qwen via HuggingFace) to parse prescription images into structured medicine data. This data drives an intelligent alarm generation system backed by a local SQLite database and Android's AlarmManager.

### 2.2 Modules of the System

- **Prescription Scanning Module** captures or imports a prescription image and submits it to the OCR pipeline.
- **OCR & AI Extraction Module** processes the image through PaddleOCR and an LLM to produce structured JSON medicine data.
- **Medicine Verification Module** presents extracted data to the user in editable cards; highlights low-confidence fields.
- **Alarm Generation Module** converts confirmed medicine schedules into precise alarm times using frequency and meal-relation rules.
- **Alarm Scheduling Module** registers alarms with AlarmManager; handles repeating, one-shot, and boot-rescheduling scenarios.

### 2.3 Workflow of the System

1. User photographs or selects a prescription image.
2. Image is uploaded to the PaddleOCR engine; job is polled until complete.
3. Extracted text is forwarded to the Qwen LLM with a strict medical extraction prompt.
4. LLM returns structured JSON containing medicine name, type, frequency, timing, meal relation, confidence, and more.

5. User reviews and edits the data on the Verification screen.
6. Confirmed medicines are saved to the SQLite database.
7. Alarm times are calculated and scheduled via AlarmManager.
8. At trigger time, AlarmReceiver fires, AlarmService plays sound and shows a full-screen notification.
9. User marks dose as Taken or Snoozes; log is updated in the database.

## 3. System Architecture

### 3.1 Architecture Overview

MediRemind follows a **layered modular architecture** where each functional concern is separated into a distinct layer. This structure improves maintainability, enables independent testing of components, and provides a clear data flow from user input to alarm delivery.

### 3.2 Architecture Diagram


### 3.3 Main Components

- **User Interface Layer**: All Activities and XML layouts that handle user interaction: scanning, verification, dashboard, and alarm ring screen.
- **Processing Layer**: The `OcrPipelineClient` manages image upload, job polling,

JSONL result parsing, and LLM submission. `MedicineJsonParser` converts raw JSON into `Medicine` model objects.

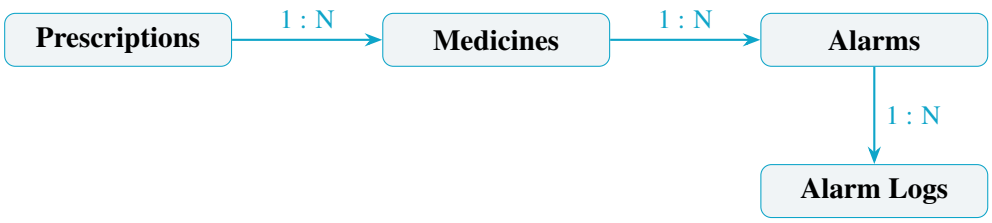
- **Business Logic Layer**: `AlarmCalculator` maps frequency types and timing hints to exact epoch-millisecond alarm times, applying meal-relation offsets. `AlarmScheduler` wraps Android's AlarmManager with retry-safe exact alarm scheduling.
- **Database Layer**: `DatabaseHelper` (SQLiteOpenHelper) manages all four tables with foreign key constraints and provides typed CRUD methods for each entity.
- **Android System Layer**: `AlarmReceiver` (BroadcastReceiver) handles alarm triggers; `AlarmService` (ForegroundService) plays ringtone and vibration; `BootReceiver` reschedules all alarms after device restart.

## 4. Database Design

### 4.1 Overview

MediRemind uses **SQLite** as its local, offline database. The schema consists of four related tables connected through foreign keys, ensuring referential integrity across prescriptions, medicines, alarms, and alarm history.

### 4.2 Entity Relationship



### 4.3 Tables

**Prescriptions**

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| patient_name | TEXT | Patient name |
| doctor_name | TEXT | Doctor name (optional) |
| date | TEXT | Prescription date |
| image_path | TEXT | Local path to captured image |
| follow_up | TEXT | Follow-up instruction from Prescription |
| created_at | TEXT | Record creation timestamp |

**Medicines**

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| prescription_id | INTEGER FK | Links to Prescriptions.id |
| medicine_name | TEXT | Full name including prefix |
| medicine_type | TEXT | tablet / capsule / syrup / etc. |
| strength | TEXT | e.g. 500mg |
| dose_quantity | INTEGER | Number of units per dose |
| frequency_type | TEXT | daily / tds / morning_evening / etc. |
| timing_hint | TEXT | morning / evening / night / etc. |
| meal_relation | TEXT | before_meal / after_meal / empty_stomach |
| special_instruction | TEXT | Additional usage instructions |
| duration_days | INTEGER | Course length in days |
| stat_dose | INTEGER | 1 if immediate first dose required |
| confidence | REAL | AI confidence score (0.0 – 1.0) |
| is_active | INTEGER | 1 if alarm still running |
| is_confirmed | INTEGER | 1 after user verification |

**Alarms**

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| medicine_id | INTEGER FK | Links to Medicines.id |
| medicine_name | TEXT | Denormalized for quick display |
| alarm_time | INTEGER | Epoch milliseconds for AlarmManager |
| repeat_every_days | INTEGER | 0 = one-shot, 1 = daily, 14 = fortnightly |
| is_enabled | INTEGER | 1 if alarm is active |
| last_triggered | INTEGER | Epoch ms of last trigger |

**Alarm Logs**

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| alarm_id | INTEGER FK | Links to Alarms.id |
| medicine_id | INTEGER | Links to Medicines.id |
| medicine_name | TEXT | Denormalized name |
| scheduled_time | INTEGER | Planned trigger time |

## 5. Alarm System Logic

### 5.1 Overview

The alarm system is the core functional component of MediRemind. It translates medicine schedule data into precise Android alarms, ensuring doses are never missed regardless of device state.

### 5.2 Frequency to Time Mapping

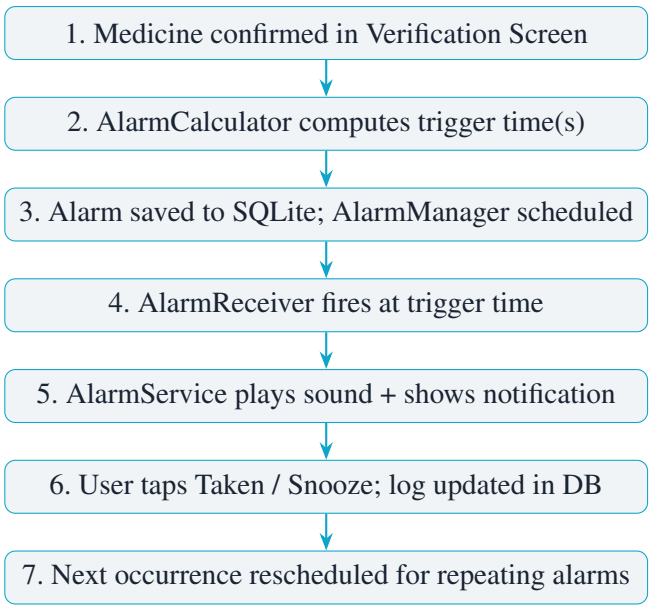
Alarm times are computed from the `frequency_type` and `timing_hint` fields extracted by the AI:

| Rule | Alarm Time(s) |
|---|---|
| daily + morning | 08:00 |
| daily + afternoon | 14:00 |
| daily + evening | 20:00 |
| daily + night | 21:00 |
| daily + null | User selects via Time Picker |
| morning_evening | 08:00 and 20:00 |
| tds | 08:00, 14:00, and 20:00 |
| night_only | 21:00 |
| sos | No alarm generated |

### 5.3 Meal Relation Adjustment

| Meal Relation | Adjustment |
|---|---|
| before_meal | Base time − 30 minutes |
| after_meal | Base time + 30 minutes |
| empty_stomach | Fixed override to 07:30 |

### 5.4 Alarm Execution Flow



### 5.5 Reliability Features

- **Boot Recovery**: `BootReceiver` listens for `BOOT_COMPLETED` and reschedules all active alarms from SQLite.
- **Snooze**: Alarm can be snoozed for 10 minutes; a one-shot re-trigger is scheduled without disturbing the main repeat cycle.
- **Full-Screen Intent**: `AlarmRingActivity` is launched over the lock screen so the user sees the alert even when the phone is idle.

## 6. Implementation

### 6.1 Technology Stack

| Component | Technology |
|---|---|
| Language | Java (Android SDK 26+) |
| UI | XML Layouts, Material Design 3 |
| Database | SQLite via SQLiteOpenHelper |
| OCR Engine | PaddleOCR-VL-1.6 (cloud API) |
| AI Extraction | Qwen3.6-7B-Instruct Inference via HuggingFace Router |
| Alarm Scheduling | Android AlarmManager |
| Alarm Delivery | ForegroundService + BroadcastReceiver |
| IDE | Android Studio |

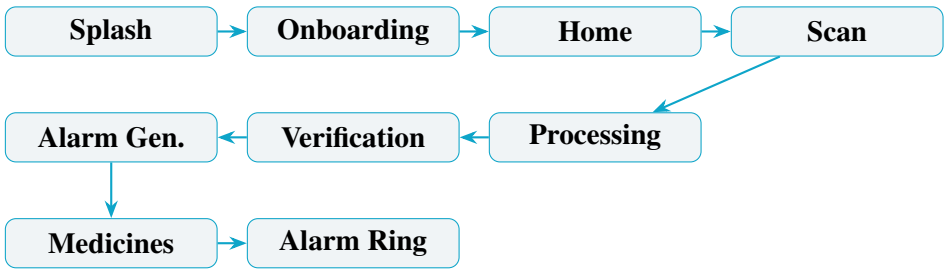
### 6.2 Key Java Classes

| Class | Responsibility |
|---|---|
| `OcrPipelineClient` | Upload image, poll OCR job, call LLM |
| `MedicineJsonParser` | Parse LLM JSON into Medicine objects |
| `AlarmCalculator` | Compute exact epoch-ms trigger times |
| `AlarmScheduler` | Wrap AlarmManager; schedule / cancel |
| `AlarmReceiver` | BroadcastReceiver; write log, start service |
| `AlarmService` | ForegroundService; play sound + notify |
| `BootReceiver` | Reschedule all alarms after reboot |
| `DatabaseHelper` | SQLiteOpenHelper for all 4 tables |
| `MedicineVerificationAdapter` | Editable RecyclerView cards with confidence UI |

## 7. User Interface

### 7.1 Screen Flow

The application follows a linear, guided flow for first-time users and provides direct access to the dashboard for returning users.




### 7.2 Screens Description

- **Splash Screen**: Hero gradient background with app logo and animated fade-in. Routes to Onboarding (first launch) or Home (returning).
- **Onboarding** : Soft mint background, floating chip badges, ViewPager2 swipe navigation with animated progress dots.
- **Home Screen**: Today's medicine schedule, upcoming alarm count, and FAB to scan a

new prescription.

- **Scan Screen**: Camera capture or gallery selection with FileProvider integration.
- **Processing Screen**: Live status messages and indeterminate progress bar while OCR and LLM pipeline runs on a background thread.
- **Verification Screen**: Editable medicine cards; low-confidence fields highlighted in amber; time picker for medicines with no detected timing.
- **Alarm Generation Screen**: Confirmation list of all scheduled alarms with time, repeat cycle, and meal relation.
- **Dashboard Screen**: Day timeline with Taken / Missed / Upcoming statistics and inline action buttons.
- **Alarm Ring Screen**: Full-screen activity shown over the lock screen with medicine name, dosage,meal relation and Taken / Snooze buttons.

## 8. Conclusion

MediRemind successfully demonstrates the integration of AI-based prescription parsing with a robust Android alarm scheduling system. The application addresses a genuine healthcare problem missed medication doses through a practical, user-friendly mobile solution.

The project applies core concepts from both *Artificial Intelligence* (OCR-based extraction, LLM-based structured parsing) and *Mobile Application Development* (Activities, Services, BroadcastReceivers, SQLite, AlarmManager) in a real-world context. The modular architecture ensures that each component can be maintained, tested, and extended independently.

Future enhancements may include cloud synchronization, medication interaction checking, caregiver notifications, and support for additional prescription languages and formats.